

I. Pen and paper

x_i	y_1	y_2	y_{mean}	y_{dev}
x_1	2	2	3.5	A
x_2	1	1	1.0	A
x_3	3	2	3.8	B
x_4	6	3	10.1	C
x_5	8	1	8.5	C

$$\phi(y_1, y_2) = y_1 \times y_2$$

$$\phi_1 = 2 \times 2 = 4$$

$$\phi_2 = 1$$

$$\phi_3 = 6$$

$$\phi_4 = 18$$

$$\phi_5 = 8$$

1.

①

$$\hat{z} = w_0 + w_1 x$$

$$w = (X^T X)^{-1} X^T z$$

$$X = \begin{bmatrix} 1 & 4 \\ 1 & 1 \\ 1 & 6 \\ 1 & 18 \\ 1 & 8 \end{bmatrix}$$

$$z = \begin{bmatrix} 3.5 \\ 1.0 \\ 3.8 \\ 10.1 \\ 8.5 \end{bmatrix}$$

$$X^T = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 4 & 1 & 6 & 18 & 8 \end{bmatrix}$$

$$X^T \cdot X = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 4 & 1 & 6 & 18 & 8 \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 1 & 1 \\ 1 & 6 \\ 1 & 18 \\ 1 & 8 \end{bmatrix} = \begin{bmatrix} 5 & 37 \\ 37 & 441 \end{bmatrix}$$

$$\det(X^T \cdot X) = 836$$

$$(X^T \cdot X)^{-1} = \begin{bmatrix} 0,52751 & -0,04426 \\ -0,04426 & 0,00598 \end{bmatrix}$$

$$W = \begin{bmatrix} 0,52751 & -0,04426 \\ -0,04426 & 0,00598 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 4 & 1 & 6 & 18 & 8 \end{bmatrix} \cdot \begin{bmatrix} 3,5 \\ 1,0 \\ 3,8 \\ 10,1 \\ 8,5 \end{bmatrix} =$$

$$= \begin{bmatrix} 1,46084 \\ 0,52925 \end{bmatrix}$$

$$\hat{z} = 1,46084 + 0,52925 x$$

2.

$$\textcircled{2} \lambda = 1$$

$$W = (X^T X + \lambda I)^{-1} X^T z$$

$$X^T X + \lambda I = \begin{bmatrix} 5 & 37 \\ 37 & 441 \end{bmatrix} + 1 \times \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 6 & 37 \\ 37 & 442 \end{bmatrix}$$

$$\det(X^T X + \lambda I) = 1283$$

$$(X^T X + \lambda I)^{-1} = \begin{bmatrix} 0,3445 & -0,02884 \\ -0,02884 & 0,00468 \end{bmatrix}$$

$$W = \begin{bmatrix} 0,3445 & -0,02884 \\ -0,02884 & 0,00468 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 4 & 1 & 6 & 18 & 8 \end{bmatrix} \begin{bmatrix} 3,5 \\ 1,0 \\ 3,8 \\ 10,1 \\ 8,5 \end{bmatrix} =$$

$$= \begin{bmatrix} 0,97267 \\ 0,57017 \end{bmatrix}$$

$$\hat{z}^1 = 0,97267 + 0,57017 x$$

	w_0	w_1
OLS	1,46074	0,52925
Ridge	0,97267	0,57017
Diferença	-0,48807	0,04092

w_0 reduziu em cerca de 33%. enquanto que w_1 aumentou em cerca de 8%.

A regularização de Ridge ($\lambda=1$) adiciona uma penalização Proporcional ao quadrado dos coeficientes. Isto força o modelo a preferir coeficientes mais pequenos, resultando num modelo mais simples e robusto. Observamos que w_0 diminuiu significativamente enquanto w_1 aumentou ligeiramente, mostrando que o Ridge redistribuiu os pesos para minimizar a sua magnitude total, reduzindo o risco de overfitting.

3.

$$x_6 = (0, 2, 1)$$

$$x_7 = (3, 4, 6.2)$$

$$x_8 = (5, 1, 3.6)$$

$$\phi_6 = 0$$

$$y_{\text{mean}_6} = 1$$

$$\phi_7 = 12$$

$$y_{\text{mean}_7} = 6.2$$

$$\phi_8 = 5$$

$$y_{\text{mean}_8} = 3.6$$

OLS

Trains

$$Z = \begin{bmatrix} 3.5 \\ 4.0 \\ 3.8 \\ 10.1 \\ 8.5 \end{bmatrix}$$

$$\hat{Z} = \begin{bmatrix} 3.57784 \\ 4.99003 \\ 4.63634 \\ 10.98734 \\ 5.69484 \end{bmatrix}$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |z_i - \hat{z}_i|$$

$$= \frac{1}{5} \times \left(|3.5 - 3.57784| + |4.0 - 4.99003| + |3.8 - 4.63634| + |10.1 - 10.98734| + |8.5 - 5.69484| \right)$$

$$= 1.11935$$

Teste

$$\hat{z} = \begin{bmatrix} 1 \\ 6,2 \\ 3,6 \end{bmatrix}$$

$$\hat{z} = \begin{bmatrix} 1,46084 \\ 7,81184 \\ 4,10709 \end{bmatrix}$$

$$\begin{aligned} MAE &= \frac{1}{3} \times (|1 - 1,46084| + |6,2 - 7,81184| + |3,6 - 4,10709|) \\ &= 0,95992 \end{aligned}$$

Ridge

Trans

$$\hat{z} = \begin{bmatrix} 3,5 \\ 1,0 \\ 3,8 \\ 10,1 \\ 8,5 \end{bmatrix}$$

$$\hat{z} = \begin{bmatrix} 3,25335 \\ 1,54284 \\ 4,39369 \\ 11,23573 \\ 5,53403 \end{bmatrix}$$

$$\begin{aligned} MAE &= \frac{1}{5} \times (|3,5 - 3,25335| + |1 - 1,54284| + |3,8 - 4,39369| + \\ &\quad + |10,1 - 11,23573| + |8,5 - 5,53403|) \\ &= 1,09697 \end{aligned}$$

Teste

$$\hat{z} = \begin{bmatrix} 1 \\ 6,2 \\ 3,6 \end{bmatrix}$$

$$\hat{z} = \begin{bmatrix} 0,97267 \\ 7,81471 \\ 3,82352 \end{bmatrix}$$

$$\begin{aligned} MAE &= \frac{1}{3} \times (|1 - 0,97267| + |6,2 - 7,81471| + |3,6 - 3,82352|) \\ &= 0,62185 \end{aligned}$$

	MSE Treino	MSE Teste	Diferença (Teste - Treino)
OLS	1,11935	0,85992	- 0,25943
Ridge	1,09697	0,62185	- 0,53512

Comparando o desempenho dos dois modelos, verifica-se que o modelo de Ridge apresenta um MSE de Treino ligeiramente inferior (1,09697 vs 1,11935) e um MSE de Teste significativamente melhor (0,62185 vs 0,85992) representando uma melhoria de cerca de 28% face ao modelo OLS.

Estes resultados confirmam que a regularização de Ridge melhorou a capacidade de generalização do modelo. Embora ambos apresentem erro de teste inferior ao de treino (situação atípica mas justificável em conjuntos de teste muito pequenos), o modelo de Ridge demonstrou melhor desempenho em dados não vistos, reduzindo o risco de overfitting como esperado.

4.

Input: 2 features (x_1, x_2)

Hidden layer: 3 neurónios ($W^{[1]}$ e 3×2)

Output layer: 3 neurónios (classes A, B, C)

$$W^{[1]} = \begin{bmatrix} 0,1 & 0,1 \\ 0,1 & 0,2 \\ 0,2 & 0,1 \end{bmatrix} \quad b^{[1]} = \begin{bmatrix} 0,1 \\ 0 \\ 0,1 \end{bmatrix}$$

$$W^{(2)} = \begin{bmatrix} 1 & 2 & 2 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad b^{(2)} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

Class A $\rightarrow$ one hot encoding $t = [1, 0, 0]$

Class B $\rightarrow t = [0, 1, 0]$

Class C $\rightarrow t = [0, 0, 1]$

Given input x_1 :

Input: $y_1 = 2$; $y_2 = 2$

Target: class A $\rightarrow t = [1, 0, 0]$

Forward Propagation

$$z^{(1)} = V^{(1)} \times X^{(0)} + b^{(1)}$$

$$z^{(1)} = \begin{bmatrix} 0,1 & 0,1 \\ 0,1 & 0,2 \\ 0,2 & 0,1 \end{bmatrix} \times \begin{bmatrix} 2 \\ 2 \end{bmatrix} + \begin{bmatrix} 0,1 \\ 0 \\ 0,1 \end{bmatrix} = \begin{bmatrix} 0,5 \\ 0,6 \\ 0,7 \end{bmatrix}$$

$$X^{(1)} = \sigma(z^{(1)}) = \begin{bmatrix} 0,62246 \\ 0,64566 \\ 0,66819 \end{bmatrix}$$

$$z^{(2)} = W^{(2)} \times X^{(1)} + b^{(2)}$$

$$z^{(2)} = \begin{bmatrix} 1 & 2 & 2 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix} \times \begin{bmatrix} 0,62246 \\ 0,64566 \\ 0,66819 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4,25016 \\ 3,58137 \\ 2,93631 \end{bmatrix}$$

$$X^{(2)} = \text{softmax}(z^{(2)}) = \frac{e^{x_i}}{\sum_{k=1}^n e^{x_k}}$$

$$e^{4,25016} = 70,11663$$

$$e^{3,58197} = 35,94428$$

$$e^{2,93631} = 18,84617$$

$$\sum_{i=1}^m x_i = 70,11663 + 35,94428 + 18,84617 = 124,90708$$

$$X^{[2]} = \begin{bmatrix} 0,56135 \\ 0,28777 \\ 0,15088 \end{bmatrix}$$

Cross-Entropy loss:

$$L = - \sum_{c=1}^m (t(c) \times \log(x^{(2)}(c)))$$

$$t = [1, 0, 0]$$

$$L = - (1 \times \log(0,56135) + 0 \times \log(0,28777) + 0 \times \log(0,15088))$$

$$= 0,57741$$

Backward Propagation

$$\frac{dz^{(i)}}{dw^{(i)}} = x^{(i-1)}$$

$$v_0 = 0,5$$

$$\frac{dz^{(i)}}{dx^{(i-1)}} = w^{(i)}$$

$$\frac{dL}{dx^{(2)}} = x^{(2)} - t$$

$$S^{(1)} = X^{(2)} - T = \begin{bmatrix} 0,56135 \\ 0,28777 \\ 0,15088 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -0,43865 \\ 0,28777 \\ 0,15088 \end{bmatrix}$$

$$\frac{dE}{dw^{(2)}} = S^{(2)} (X^{(1)})^T = \begin{bmatrix} -0,43865 \\ 0,28777 \\ 0,15088 \end{bmatrix} \begin{bmatrix} 0,62246 & 0,61566 & 0,66819 \end{bmatrix}$$

$$= \begin{bmatrix} -0,27304 & -0,28322 & -0,29310 \\ 0,17912 & 0,18580 & 0,19228 \\ 0,09392 & 0,09742 & 0,10082 \end{bmatrix}$$

$$W_{new}^{(1)} = \begin{bmatrix} 1 & 2 & 2 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix} - 0,5 \times \begin{bmatrix} -0,27304 & -0,28322 & -0,29310 \\ 0,17912 & 0,18580 & 0,19228 \\ 0,09392 & 0,09742 & 0,10082 \end{bmatrix}$$

$$= \begin{bmatrix} 1,13652 & 2,14161 & 2,14655 \\ 0,91044 & 1,90710 & 0,90386 \\ 0,95304 & 0,95129 & 0,94959 \end{bmatrix}$$

$$\frac{dE}{db^{(1)}} = S^{(2)} = \begin{bmatrix} -0,43865 \\ 0,28777 \\ 0,15088 \end{bmatrix}$$

$$b_{new}^{(1)} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - 0,5 \begin{bmatrix} -0,43865 \\ 0,28777 \\ 0,15088 \end{bmatrix} = \begin{bmatrix} 1,21932 \\ 0,85611 \\ 0,92456 \end{bmatrix}$$

$$\delta^{(1)} = \left((V^{(1)})^T \cdot \delta^{(2)} \right) \odot \frac{dx^{(1)}}{dz^{(1)}}$$

$$\frac{dx^{(1)}}{dz^{(1)}} = d(\sigma(z^{(1)})) = x^{(1)}(1-x^{(1)})$$

$$\delta^{(1)} = \left(\begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 1 \\ 2 & 1 & 1 \end{bmatrix} \begin{bmatrix} -0,43865 \\ 0,28777 \\ 0,45088 \end{bmatrix} \right) \odot \left(\begin{bmatrix} 0,62246 \\ 0,64566 \\ 0,66819 \end{bmatrix} \odot \left(1 - \begin{bmatrix} 0,62246 \\ 0,64566 \\ 0,66819 \end{bmatrix} \right) \right)$$

$$\delta^{(1)} = \begin{bmatrix} 0 \\ -0,15089 \\ -0,43965 \end{bmatrix} \odot \begin{bmatrix} 0,2350 \\ 0,22878 \\ 0,22171 \end{bmatrix} = \begin{bmatrix} 0 \\ -0,03452 \\ -0,09725 \end{bmatrix}$$

$$\frac{dE}{dW^{(1)}} = \delta^{(1)} (x^{(2)})^T = \begin{bmatrix} 0 \\ -0,03452 \\ -0,09725 \end{bmatrix} \begin{bmatrix} 2 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0,06904 \\ -0,06904 & -0,1945 \end{bmatrix}$$

$$W_{new}^{(1)} = \begin{bmatrix} 0,1 & 0,1 \\ 0,1 & 0,2 \\ 0,2 & 0,1 \end{bmatrix} - 0,5 \begin{bmatrix} 0 & 0,06904 \\ -0,06904 & -0,1945 \\ -0,1945 & -0,1945 \end{bmatrix}$$

$$W_{new}^{(1)} = \begin{bmatrix} 0,1 & 0,1 \\ 0,13452 & 0,23452 \\ 0,29725 & 0,19725 \end{bmatrix}$$

$$\frac{dE}{db^{(1)}} = \delta^{(1)}$$

$$b_{new}^{[1]} = \begin{bmatrix} 0,1 \\ 0 \\ 0,1 \end{bmatrix} - 0,5 \times \begin{bmatrix} 0 \\ -0,03452 \\ -0,09725 \end{bmatrix}$$

$$b_{new}^{[1]} = \begin{bmatrix} 0,1 \\ 0,01726 \\ 0,14862 \end{bmatrix}$$

A diferença fundamental na sigmoid vs. sem ativação está na capacidade representacional:

- Sem ativação → O modelo torna-se puramente linear.

Independente do número de camadas, o resultado é equivalente a uma única transformação linear, limitando o modelo a aprender apenas fronteiras de decisão lineares.

- Com sigmoid → A não-linearidade permite ao modelo aprender padrões complexos e fronteiras de decisão não-lineares, aumentando significativamente a sua capacidade de representar relações complexas nos dados.

II. Programming

5.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error, make_scorer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns

# Carregar o dataset
df = pd.read_csv('rent.csv')

# Separar features e target
X = df[['bathrooms', 'bedrooms', 'longitude', 'latitude']]
y = df['price']

# Definir o scorer para MAE
mae_scorer = make_scorer(mean_absolute_error, greater_is_better=False)

# Modelos
lin_reg = LinearRegression()
mlp_no_activation = make_pipeline(
    StandardScaler(),
    MLPRegressor(hidden_layer_sizes=(5, 5),
                  activation='identity',
                  max_iter=200,
                  random_state=0)
)

mlp_relu = make_pipeline(
    StandardScaler(),
    MLPRegressor(hidden_layer_sizes=(5, 5),
                  activation='relu',
                  max_iter=200,
                  random_state=0)
)

# Avaliar com 5-fold cross-validation
lin_reg_scores = cross_val_score(lin_reg, X, y, cv=5, scoring=mae_scorer)
mlp_no_activation_scores = cross_val_score(mlp_no_activation, X, y, cv=5, scoring=mae_scorer)
mlp_relu_scores = cross_val_score(mlp_relu, X, y, cv=5, scoring=mae_scorer)

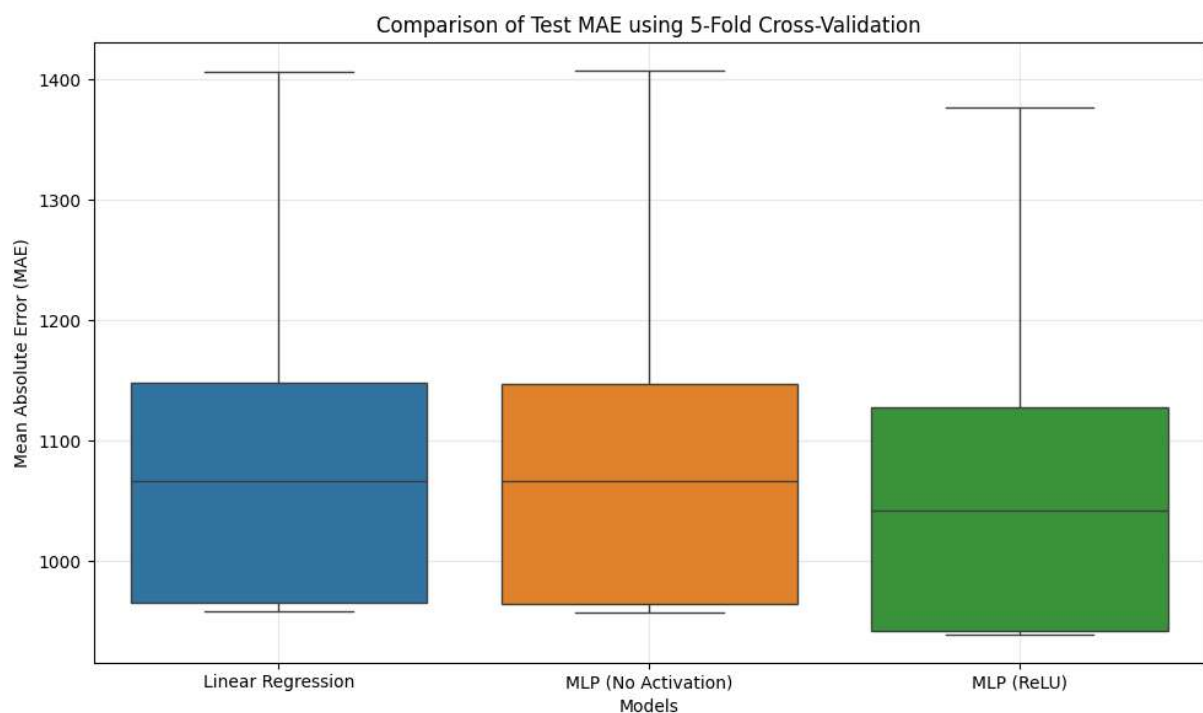
# Converter para MAE positivo
lin_reg_errors = -lin_reg_scores
mlp_no_activation_errors = -mlp_no_activation_scores
mlp_relu_errors = -mlp_relu_scores

# Criar boxplots
plt.figure(figsize=(10, 6))
errors_df = pd.DataFrame({
    'Linear Regression': lin_reg_errors,
    'MLP (No Activation)': mlp_no_activation_errors,
    'MLP (ReLU)': mlp_relu_errors
})

sns.boxplot(data=errors_df)
plt.title('Comparison of Test MAE using 5-Fold Cross-Validation')
plt.ylabel('Mean Absolute Error (MAE)')
plt.xlabel('Models')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

✓ 2m 45.6s

Python



6.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error, make_scorer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns

# Carregar o dataset
df = pd.read_csv("rent.csv")

# Separar features e target
X = df[['bathrooms', 'bedrooms', 'longitude', 'latitude']]
y = df['price']

# Inicializar listas para armazenar os MAEs
mae_linreg = []
mae_mlp_relu = []

# Fazer 10 divisões diferentes de treino e teste
for i in range(1, 11):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=i)

    # Modelo 1: Regressão Linear
    linreg = LinearRegression()
    linreg.fit(X_train, y_train)
    pred_linreg = linreg.predict(X_test)
    mae_linreg.append(mean_absolute_error(y_test, pred_linreg))

    # Modelo 2: MLP com ReLU
    mlp_relu = make_pipeline(
        StandardScaler(),
        MLPRegressor(hidden_layer_sizes=(5, 5),
                      activation='relu',
                      max_iter=200,
                      random_state=0)
    )
    mlp_relu.fit(X_train, y_train)
    pred_mlp_relu = mlp_relu.predict(X_test)
    mae_mlp_relu.append(mean_absolute_error(y_test, pred_mlp_relu))

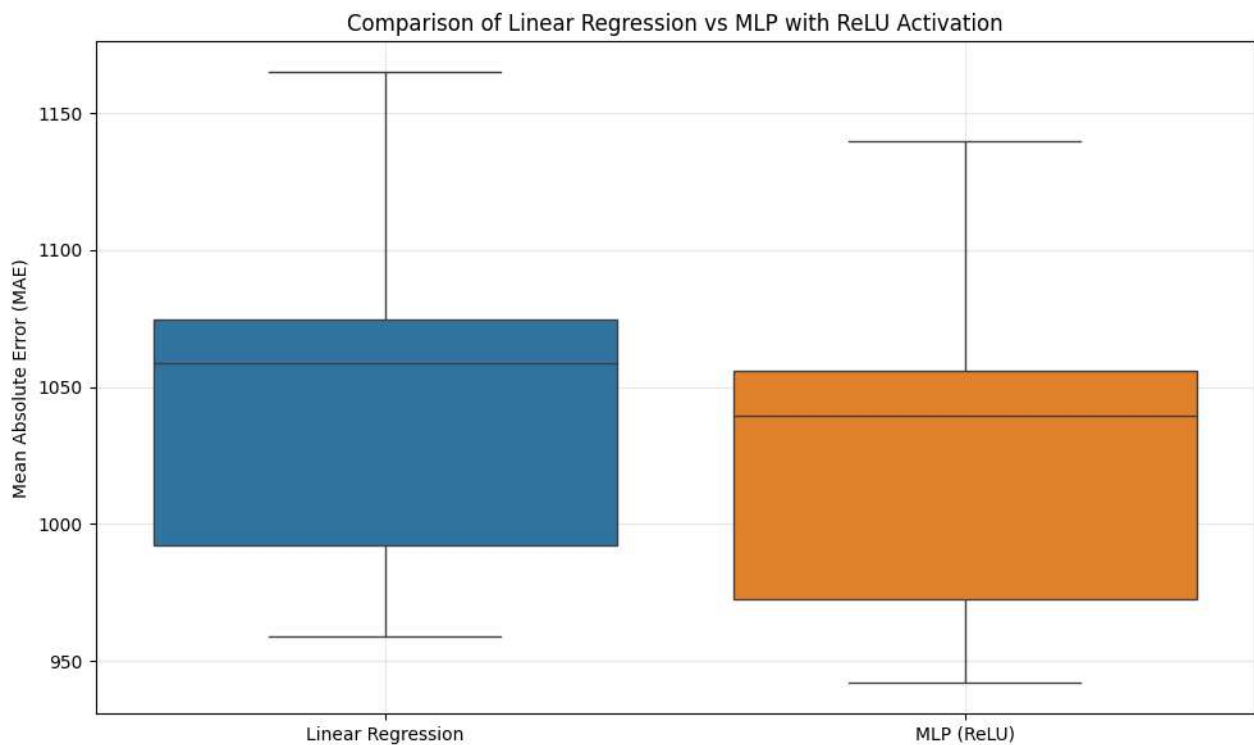
# Preparar dados para boxplot
mae_df = pd.DataFrame({
    'Linear Regression': mae_linreg,
    'MLP (ReLU)': mae_mlp_relu
})

# Plot do boxplot
plt.figure(figsize=(10, 6))
sns.boxplot(data=mae_df)
plt.title("Comparison of Linear Regression vs MLP with ReLU Activation")
plt.ylabel("Mean Absolute Error (MAE)")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Estatísticas descritivas para análise
print("Estatísticas descritivas do MAE para cada modelo:")
print(mae_df.describe())
```

✓ 3m 58.1s

Python



Comparando os erros (via boxplot ou estatísticas descritivas), observamos que a Regressão Linear apresenta um MAE médio de aproximadamente 1054, enquanto o MLP com ReLU apresenta um MAE médio de cerca de 1020.

Neste caso, o MLP com ativação ReLU apresenta um desempenho ligeiramente superior ao da regressão linear, com uma redução de aproximadamente 34 no MAE. Este resultado demonstra o impacto positivo das funções de ativação não-lineares, permitindo ao MLP capturar relações mais complexas nos dados. A diferença, embora modesta, indica que existem algumas não-linearidades no dataset que o MLP consegue modelar melhor do que a regressão linear.

É importante destacar que o uso de normalização (StandardScaler) foi crucial para o bom desempenho do MLP. As redes neurais são sensíveis à escala das features, e a normalização permite que o modelo convirja de forma mais eficiente e estável. Sem normalização, o MLP teria dificuldade em aprender padrões relevantes.

7.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import KFold
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# Carregar o dataset
df = pd.read_csv('rent.csv')

# Separar features e target
X = df[['bathrooms', 'bedrooms', 'longitude', 'latitude']].values
scaler = StandardScaler()
scaler.fit(X)
X = scaler.transform(X)
y = df['price'].values

# Configurar 5-fold cross-validation
kfold = KFold(n_splits=5, shuffle=True, random_state=42)

# Listas para armazenar as perdas de treino e validação por iteração
train_losses_all_folds = []
val_losses_all_folds = []

# Para cada fold
for fold, (train_idx, val_idx) in enumerate(kfold.split(X)):
    print(f"Processing fold {fold + 1}/5...")

    X_train, X_val = X[train_idx], X[val_idx]
    y_train, y_val = y[train_idx], y[val_idx]

    # Criar o MLP com ReLU
    mlp = make_pipeline(
        StandardScaler(),
        MLPRegressor(
            hidden_layer_sizes=(5, 5),
            activation='relu',
            max_iter=300,
            random_state=0,
            warm_start=True,
            learning_rate_init=0.001)
    )

    train_losses = []
    val_losses = []

    # Treinar iterativamente (guardar a cada 10 iterações)
    for iteration in range(10, 301, 10):
        mlp.max_iter = iteration
        mlp.fit(X_train, y_train)

        # Calcular loss (MAE) para treino e validação
        train_pred = mlp.predict(X_train)
        val_pred = mlp.predict(X_val)

        train_loss = mean_absolute_error(y_train, train_pred)
        val_loss = mean_absolute_error(y_val, val_pred)

        train_losses.append(train_loss)
        val_losses.append(val_loss)

    train_losses_all_folds.append(train_losses)
    val_losses_all_folds.append(val_losses)

# Calcular a média das perdas através dos folds
avg_train_losses = np.mean(train_losses_all_folds, axis=0)
avg_val_losses = np.mean(val_losses_all_folds, axis=0)

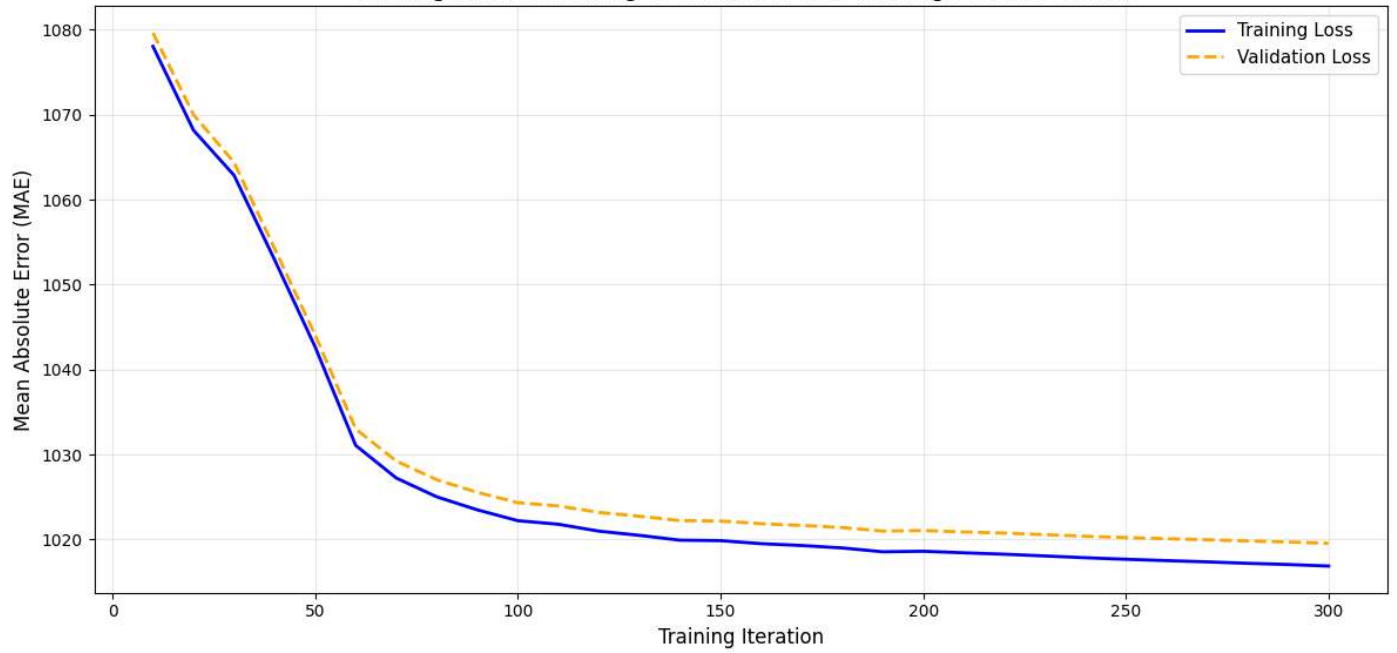
# Plot das curvas de aprendizagem com melhor visualização
iterations = list(range(10, 301, 10))
plt.figure(figsize=(12, 6))
plt.plot(iterations, avg_train_losses, label='Training Loss', linewidth=2, color='blue', linestyle='-')
plt.plot(iterations, avg_val_losses, label='Validation Loss', linewidth=2, color='orange', linestyle='--')
plt.xlabel('Training Iteration', fontsize=12)
plt.ylabel('Mean Absolute Error (MAE)', fontsize=12)
plt.title('Learning Curves: Training vs Validation Loss (Average across 5 folds)', fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Mostrar os valores finais
print(f"Final Training MAE: {avg_train_losses[-1]:.2f}")
print(f"Final Validation MAE: {avg_val_losses[-1]:.2f}")
print(f"Difference: {abs(avg_val_losses[-1] - avg_train_losses[-1]):.2f}")
```

✓ 10m 42.3s

Python

Learning Curves: Training vs Validation Loss (Average across 5 folds)



```
Final Training MAE: 1016.87
Final Validation MAE: 1019.54
Difference: 2.67
```

Analisando as curvas de aprendizagem obtidas, observamos que tanto o MAE de treino como o MAE de validação apresentam um comportamento muito semelhante ao longo das iterações. Ambas as curvas diminuem rapidamente nas primeiras ~100 iterações, passando de um MAE inicial de aproximadamente 1080 para cerca de 1020, e depois estabilizam. No final do treino (iteração 300), o MAE de treino é de 1016.87 e o MAE de validação é de 1019.54, com uma diferença mínima de apenas 2.67 (correspondente a uma variação do MAE de validação de cerca de 0.3% face MAE de treino).

É possível verificar, no entanto, que um erro absoluto de cerca de 1000 representaria, em casos mais extremos, um desvio percentual máximo de cerca de 80% face ao valor real de price. Isto indica que o modelo se encontra em underfitting, uma vez que, apesar da sobreposição das curvas ser satisfatória, a magnitude relativa do erro face aos valores patentes no dataset é demasiado elevada.

Em suma, o MLP com ativação ReLU não demonstra uma capacidade de generalização adequada para este problema.